

C# Lead Developer: System Design & Architecture Cheatsheet

Interview Preparation Notes

November 29, 2025

Contents

I	System Design & Architecture	3
1	System Design Core Concepts	3
1.1	The 4 Steps of Design Interview (Initial Brainstorm)	3
2	Scalability (Key for ServiceTitan)	3
2.1	Core System Components & Metrics	3
2.2	Architectural Comparisons (Monolith vs. Microservices)	3
2.3	Key Scaling Techniques and Technologies	4
2.4	Data Persistence Models (SQL vs. NoSQL)	4
2.5	NoSQL System Types and Examples	5
2.6	The 7 Layers of the OSI Model (Context)	6
3	Concurrency & Async in C#	6
3.1	The <code>async</code> and <code>await</code> Keywords	6
3.2	Concurrency vs. Parallelism	6
II	Software Design Principles: SOLID & GRASP	7
4	SOLID Principles (Object-Oriented Design)	7
4.1	Single Responsibility Principle (SRP)	7
4.2	Open/Closed Principle (OCP)	7
4.3	Liskov Substitution Principle (LSP)	7
4.4	Interface Segregation Principle (ISP)	7
4.5	Dependency Inversion Principle (DIP)	7
5	GRASP Principles (General Responsibility Assignment)	7
5.1	Information Expert	7
5.2	Creator	8
5.3	Low Coupling	8
5.4	High Cohesion	8
5.5	Controller	8
III	Design Patterns (GoF & Distributed)	9

6	Creational Patterns (GoF)	9
6.1	Factory Method	9
6.2	Singleton	9
6.3	Builder	9
7	Structural Patterns (GoF)	10
7.1	Adapter	10
7.2	Decorator	10
7.3	Facade	11
8	Behavioral Patterns (GoF)	11
8.1	Strategy	11
8.2	Observer	11
9	Distributed & Architectural Patterns	11
9.1	CQRS (Command Query Responsibility Segregation)	11
9.2	Saga (Distributed Transactions)	11
9.3	Circuit Breaker	12
IV	C# & .NET Core Fundamentals	13
10	Dependency Injection (DI) in .NET Core	13
10.1	Inversion of Control (IoC) Container	13
10.2	Code Example: Service Registration	13
11	LINQ (Language Integrated Query)	13
11.1	Description & Key Concepts	13
12	Garbage Collection (GC)	13
12.1	The Generational Collector	13
12.2	Value Types vs. Reference Types	14

System Design & Architecture

1 System Design Core Concepts

1.1 The 4 Steps of Design Interview (Initial Brainstorm)

1. **Requirements Elicitation (Ask Questions!)**: Functional vs. Non-Functional (Scale, Latency, Consistency). Define Scope.
2. **High-Level Design (HLD)**: Draw Bounding Boxes (Components: Client, Load Balancer, API Gateway, Services, Database). Estimate Traffic (QPS).
3. **Deep Dive (LLD) & Bottlenecks**: Choose specific technologies (e.g., C#/Kestrel, PostgreSQL/Redis). Discuss ****Scalability, Availability, Reliability, Consistency****.
4. **Trade-offs & Evolution**: Discuss alternative designs (Monolith vs. Microservices, SQL vs. NoSQL) and future scaling plans (e.g., Sharding).

2 Scalability (Key for ServiceTitan)

2.1 Core System Components & Metrics

Client: The application initiating the request (e.g., Browser, Mobile App, another Microservice). It typically interacts with the **API Gateway**.

Load Balancer (L4/L7): The core traffic director, distributing requests to prevent overload on any single service instance.

API Gateway: A single entry point for all clients. Responsible for cross-cutting concerns: ****Request Routing**** (to the correct service), Authentication/Authorization, Rate Limiting, and Metrics aggregation.

Services: Independent, deployable units containing the business logic (in a Microservices architecture).

Database: Persistent storage layer (Relational or Non-Relational).

QPS (Queries Per Second) / RPS (Requests Per Second): The critical metric defining system **throughput**. High QPS/RPS demands horizontal scaling, caching, and eventual consistency for certain data.

2.2 Architectural Comparisons (Monolith vs. Microservices)

[Image of Horizontal Scaling diagram with Load Balancer]

△ **Monolith (When better)**:

- **Pros:** Simple to develop, deploy, test, and debug initially. All code is in one codebase. Ideal for small teams and applications with uncertain future complexity.
- **Cons:** Difficult to scale specific parts, slow deployment, high coupling.
- **Use Case Example:** Small internal business tools, early-stage startups.

△ **Microservices (When better - ServiceTitan Focus):**

- **Pros:** High **Fault Tolerance**, independent scalability (scale only the heavy services), faster team velocity, technology diversity (Polyglot).
- **Cons:** High operational complexity (Kubernetes, distributed tracing), complex inter-service communication (Sagas, Service Mesh), data consistency challenges.
- **Use Case Example:** Large-scale B2B platforms, e-commerce, ServiceTitan's growing suite of products.

2.3 Key Scaling Techniques and Technologies

◇ **Load Balancing (L4/L7):** Distributes incoming traffic across multiple service instances. Prevents single-point overload and increases **Availability**.

- **Layer 4 (L4) - Transport Layer:** Operates on **TCP/UDP**. Routes traffic based only on **IP addresses and ports**. Fast and simple, but lacks application context. Good for general, high-speed connection routing.
- **Layer 7 (L7) - Application Layer:** Operates on **HTTP/HTTPS**. More intelligent; routes based on request details like **URL paths, HTTP headers, or cookies**. Essential for microservice architectures (e.g., API Gateway routing).

◇ **Caching (Redis/Memcached):**

- Fast, volatile memory (In-Memory Data Store) used to store the results of expensive computations or database queries. Reduces latency and lessens the load on the primary database.
- **Redis (The Better Option):** A versatile in-memory store that offers not only simple Key-Value caching but also advanced data structures (Lists, Sets, Sorted Sets) and **Persistence**. It is frequently used for rate limiting, session management, and as a message broker. **Redis is database agnostic**; it works with any primary persistence store, including PostgreSQL, MySQL, SQL Server, and NoSQL databases.
- **TTL (Time-To-Live):** A value defining how long data can remain in the cache before it is considered stale and must be refreshed from the main source (DB). It controls **data staleness**.

◇ **Sharding (Horizontal Partitioning):**

- A technique for scaling **RDBMS** (Relational Database Management System) by physically splitting tables across multiple database instances.
- **Shard Key:** A chosen column (e.g., **CustomerId** or **TenantId**) that determines which server (shard) a given row will be stored on.
- **Trade-off:** Improves **Write Scaling**, but complicates queries spanning multiple shards (Distributed Joins).

2.4 Data Persistence Models (SQL vs. NoSQL)

△ **SQL/RDBMS (Relational Database Management System - When better):**

- **Model:** Fixed schema, tables with predefined relations (joins).

- **Consistency:** Provides **Strong Consistency** (ACID: Atomicity, Consistency, Isolation, Durability).
- **Use Case Example:** Financial transactions, inventory management, user authentication—any domain requiring strict data integrity and complex, transactional read/write operations.

△ NoSQL/Distributed (Non-Relational - When better):

- **Model:** Flexible/dynamic schema (e.g., JSON documents, Key-Value pairs). High horizontal scalability.
- **Consistency:** Usually favors **Availability and Partition Tolerance (BASE: Basically Available, Soft state, Eventually consistent)**.
- **Why Better:** Ideal for managing massive scale, high throughput (QPS), and data with evolving schema or weak relationships.
- **Use Case Example (Specifics):**
 1. **Massive Read/Write Volume (Telemetry/IoT):** Storing millions of user events, telemetry data, or session information.
 2. **High Availability:** Storing social media feeds or user profiles where downtime is unacceptable.
 3. **Schema Flexibility:** Content Management Systems or configurations where the data structure changes frequently.

2.5 NoSQL System Types and Examples

1. Key-Value Stores (e.g., Redis, Memcached, DynamoDB):

- **Structure:** Simplest model. Data stored as a collection of **key-value pairs**. Extremely fast lookups based on the key.
- **How they Scale:** Trivial to horizontally partition (shard) based on the hash of the key.
- **Best For:** Caching, session management, user profile data retrieval, feature flags.

2. Document Databases (e.g., MongoDB, Cosmos DB):

- **Structure:** Stores data as semi-structured documents (usually ****JSON/BSON****). Documents can contain nested fields, lists, and do not require a rigid schema (schema-less).
- **How they Scale:** Documents are typically sharding units. Allows complex queries within a single document.
- **Best For:** Content Management, product catalogs, user-generated content, systems requiring flexible or evolving schemas.

3. Column-Family Stores (e.g., Cassandra, HBase):

- **Structure:** Optimized for data warehousing and massive scale time-series data. Data is grouped into ****column families**** rather than rows. Highly optimized for wide rows (many columns).
- **How they Scale:** Designed for linear horizontal scalability. Extremely high availability and fault tolerance across many nodes.
- **Best For:** Time-series data, high-volume sensor data, monitoring systems, financial ledger data, event logging.

4. Graph Databases (e.g., Neo4j, JanusGraph):

- **Structure:** Stores data as ****Nodes**** (entities), ****Edges**** (relationships), and ****Properties**** (information about nodes/edges).

- **How they Scale:** Optimized for traversing complex relationships, which is highly inefficient in relational databases.
- **Best For:** Social networks (friend connections), recommendation engines (product-to-product links), fraud detection, complex dependency mapping.

2.6 The 7 Layers of the OSI Model (Context)

Understanding the OSI model layers is crucial for system design and debugging networking issues.

Layer	Name	Data Unit	Key Function & Relevance to Load Balancing
L7	Application	Data	Interaction with user/application (HTTP, HTTPS, gRPC). L7 Load Balancers/API Gateways operate here.
L6	Presentation	Data	Translation, compression, encryption (SSL/TLS). SSL/TLS Termination often happens here (before L7 processing).
L5	Session	Data	Managing sessions (opening, maintaining, closing). Used by some Service Proxies.
L4	Transport	Segments	End-to-end data delivery, flow control (TCP, UDP). L4 Load Balancers operate only on IP/Port data.
L3	Network	Packets	Logical addressing and routing across networks (IP, ICMP). Handled by Routers.
L2	Data Link	Frames	Physical addressing (MAC), error control within a local network (LAN). Handled by Switches.
L1	Physical	Bits	Physical transmission of signals (cables, fiber optics). Abstraction layer in Cloud.

3 Concurrency & Async in C#

3.1 The async and await Keywords

Used to perform ****I/O-bound**** operations (API calls, DB queries) without blocking the thread, improving throughput. Essential for C# backend performance on Kestrel.

```

1 public async Task<List<Data>> GetDataAsync(int id)
2 {
3     // 1. Uwolnij w tek (Return to thread pool)
4     var user = await _db.Users.FindAsync(id);
5
6     // 2. Oczekuj na zakończenie I/O. Po powrocie wznowienie
7     var order = await _apiClient.GetOrdersAsync(user.Id);
8
9     return CombineData(user, order);
10 }
```

3.2 Concurrency vs. Parallelism

- **Concurrency (C#):** Handling multiple tasks **at the same time** (time-slicing). Achieved primarily via `async/await` (I/O).
- **Parallelism (.NET):** Executing multiple tasks **simultaneously** (multiple CPU cores). Achieved via `Parallel.For`, `PLINQ`. Best for ****CPU-bound**** tasks.

Software Design Principles: SOLID & GRASP

4 SOLID Principles (Object-Oriented Design)

4.1 Single Responsibility Principle (SRP)

A class should have **only one reason to change**.

Trade-off: Leads to more classes, but each is easier to test and maintain.

4.2 Open/Closed Principle (OCP)

Software entities should be **open for extension, but closed for modification**.

Achieved via **interfaces, abstract classes, and inheritance** (Strategy pattern).

4.3 Liskov Substitution Principle (LSP)

Subclasses must be able to replace their base class without breaking the application's desirable properties (i.e., they must honor the base class's contract/behavior).

4.4 Interface Segregation Principle (ISP)

Clients should not be forced to depend upon interfaces that they do not use.

Better to have many small, specific interfaces (`IReportGenerator`) than one large, "fat" one (`IReportWorker`).

4.5 Dependency Inversion Principle (DIP)

High-level modules should not depend on **low-level modules**. Both should depend on **abstractions** (interfaces).

Key principle behind **Dependency Injection (DI)**.

5 GRASP Principles (General Responsibility Assignment)

5.1 Information Expert

Assign responsibility to the class that has the **information necessary** to fulfill the responsibility.

5.2 Creator

Assign class *B* the responsibility to create an instance of class *A* if *B* ****aggregates**** *A* or is closely related to *A*.

5.3 Low Coupling

Aim for low inter-dependency between components. ****Benefit:**** High maintainability and reusability.

5.4 High Cohesion

Aim for strong functional relation among elements within a single module or class. The class should be focused on one clear task.

5.5 Controller

Assign responsibility for handling a system event (e.g., API request) to a class that represents the entire system or a major subsystem (e.g., a dedicated **ApiController**).

Design Patterns (GoF & Distributed)

6 Creational Patterns (GoF)

6.1 Factory Method

- ★ **Explanation:** Defines an interface for creating an object, but lets subclasses decide which class to instantiate. It delegates the responsibility of instantiation to the subclass.
- ★ **Benefit:** Promotes Loose Coupling by decoupling the client code from the concrete classes it needs to instantiate.

```
1 // Factory Interface (Defines the contract)
2 public interface ILogger { void Log(string message); }
3
4 // Concrete Product
5 public class FileLogger : ILogger { /* Implementation */ }
6
7 // Creator (Defines the Factory Method)
8 public abstract class LoggerFactory
9 {
10     public abstract ILogger CreateLogger();
11     public void LogMessage(string message)
12     {
13         ILogger logger = CreateLogger(); // The factory method
14         logger.Log(message);
15     }
16 }
```

6.2 Singleton

- ★ **Explanation:** Ensures a class has only one instance and provides a global point of access to it.
- ★ **Warning:** In modern C#/.NET Core, prefer registering services with a ****Singleton lifetime**** in the Dependency Injection (DI) container over manual implementation. Manual implementation can hide dependencies and complicate testing.

6.3 Builder

- ★ **Explanation:** Separates the construction of a complex object from its representation, allowing the same construction process to create different representations. Useful when an object has many optional parameters or complex initialization steps.
- ★ **Use Case Example:** Building a complex SQL query or a large configuration object step-by-step.

```

1 // Example of building a product
2 public class Product
3 {
4     public string Name { get; set; }
5     public decimal Price { get; set; }
6     public string Description { get; set; }
7 }
8
9 public class ProductBuilder
10 {
11     private Product _product = new Product();
12
13     public ProductBuilder WithName(string name)
14     { _product.Name = name; return this; }
15
16     public ProductBuilder WithPrice(decimal price)
17     { _product.Price = price; return this; }
18
19     public Product Build()
20     { return _product; }
21 }
22 // Usage: var product = new ProductBuilder().WithName("Laptop").WithPrice(1500).Build
    ();

```

7 Structural Patterns (GoF)

7.1 Adapter

- ★ **Explanation:** Allows objects with incompatible interfaces to collaborate. It acts as a translator, wrapping an existing incompatible object (the adaptee) to expose a client-compatible interface.
- ★ **Use Case Example:** Integrating a legacy third-party service (e.g., an old XML payment API) into a modern C# system that expects a standardized JSON format.

7.2 Decorator

- ★ **Explanation:** Attaches additional responsibilities to an object dynamically and transparently (without using inheritance). It wraps the original object and forwards all calls while adding custom behavior (e.g., logging, caching) before or after the original call.
- ★ **Use Case Example:** Adding logging or metrics tracking to an existing service implementation at runtime without changing the service's core code.

```

1 public class LoggingOrderService : IOrderService
2 {
3     private readonly IOrderService _wrappedService;
4     public LoggingOrderService(IOrderService service) { _wrappedService = service; }
5
6     public void PlaceOrder(Order order)
7     {
8         Console.WriteLine($"LOG: Attempting to place order {order.Id}");
9         _wrappedService.PlaceOrder(order); // Call original service
10        Console.WriteLine($"LOG: Order {order.Id} placed successfully.");
11    }
12 }
13 // DI Usage: services.AddScoped<IOrderService, LoggingOrderService>();

```

7.3 Facade

- ★ **Explanation:** Provides a simplified, unified interface to a complex subsystem or a set of interfaces. It hides the complexity of internal service interactions.
- ★ **Use Case Example:** A single ‘CheckoutFacade’ method that orchestrates calls to the ‘InventoryService’, ‘PaymentService’, and ‘NotificationService’.

8 Behavioral Patterns (GoF)

8.1 Strategy

- ★ **Explanation:** Defines a family of interchangeable algorithms, encapsulates each one, and makes them interchangeable. The client can choose the concrete algorithm (strategy) at runtime.
- ★ **Benefit:** Follows the Open/Closed Principle (OCP) as you can add new algorithms without modifying the context class.
- ★ **Use Case Example:** Implementing different payment methods (Credit Card, PayPal, Invoice) or different tax calculation rules that all share an ‘ITaxStrategy’ interface.

8.2 Observer

- ★ **Explanation:** Defines a one-to-many dependency so that when one object (**Subject**) changes state, all dependent objects (**Observers**) are notified automatically.
- ★ **Use Case Example:** A stock trading application where a price change event (Subject) notifies all subscribed users (Observers) instantly. Used heavily in event-driven systems.

9 Distributed & Architectural Patterns

9.1 CQRS (Command Query Responsibility Segregation)

Explanation: Separates the model for **updating data (Command Model)** from the model for **reading data (Query Model)**. Commands are imperative (e.g., ‘UpdateCustomerAddress’), while Queries are idempotent (e.g., ‘GetCustomerDetails’).

Benefit: Allows the read side (Query) to be highly optimized (e.g., denormalized SQL views or NoSQL) and scaled independently from the write side (Command), addressing performance bottlenecks at scale.

Use Case Example: A system where reads vastly outnumber writes (90% reads / 10% writes), such as e-commerce product pages or user feeds.

9.2 Saga (Distributed Transactions)

Explanation: A pattern to manage a sequence of local transactions where each transaction is performed by a different microservice. If any local transaction fails, the Saga executes **Compensating Transactions** in reverse order to undo the previous changes.

Challenge: Dealing with distributed data consistency challenges when transactions span multiple databases.

Use Case Example: The full lifecycle of an order: 1) *Order Service* creates order. 2) *Inventory Service* reserves stock. 3) *Payment Service* charges the customer. If Payment fails, a Compensation Transaction in Inventory Service releases stock.

9.3 Circuit Breaker

Explanation: Prevents repeated, failing calls to a remote service. When failures cross a threshold, the breaker "opens," immediately returning an error (failing fast) and giving the failing service time to recover. This prevents cascading failures.

Implementation in C#: Often achieved using the **Polly** library, which provides resilience policies.

States: **Closed** (Normal operation), **Open** (Requests immediately fail for a duration), and **Half-Open** (Allows a test request to see if the service has recovered).

C# & .NET Core Fundamentals

10 Dependency Injection (DI) in .NET Core

10.1 Inversion of Control (IoC) Container

The built-in container manages service creation and lifetime.

Lifetimes: **Singleton** (per app), **Scoped** (per request), **Transient** (every time).

10.2 Code Example: Service Registration

```
1 // Program.cs or Startup.cs
2 builder.Services.AddSingleton<IConfigurationService, ConfigManager>();
3 builder.Services.AddScoped<IOrderService, OrderProcessor>();
4
5 // Dependency is automatically injected into the constructor:
6 public class MyController : ControllerBase
7 {
8     private readonly IOrderService _orderService;
9     public MyController(IOrderService orderService) // Injected dependency
10    {
11        _orderService = orderService;
12    }
13 }
```

11 LINQ (Language Integrated Query)

11.1 Description & Key Concepts

A powerful set of features for querying data from various sources (arrays, collections, databases via Entity Framework) using C# syntax.

```
1 // Method Syntax (C# Preferred)
2 var highValueItems = dbContext.Products
3     .Where(p => p.Price > 100)
4     .OrderByDescending(p => p.Name)
5     .Select(p => new { p.Name, p.Price });
```

12 Garbage Collection (GC)

12.1 The Generational Collector

.NET GC is generational, dividing the heap into three generations (Gen 0, Gen 1, Gen 2).

Gen 0: New, short-lived objects. Collected most frequently (fastest).

Gen 2: Long-lived objects. Collected least frequently (slowest).

12.2 Value Types vs. Reference Types

Value Types (**struct**, **int**): Stored directly on the stack (no GC overhead).

Reference Types (**class**, **string**): Stored on the heap (managed by GC).